

UNAM_LOG02.png

Universidad Nacional Autónoma de México

Ingeniería en ...

Nombre de Asignatura

FORMATO DE REPORTE

ALUMNO:
309075318

Grupo:

Semestre:
2024-II

México,CDMX. Febrero 2024

Índice

1. Introducción	2
2. Marco Teórico	3
2.1. Entorno y lenguaje de programación	3
2.2. Fundamentos y sintaxis del lenguaje	3
2.3. Utilerías y clases de uso general	3
2.4. Clases y objetos	3
3. Desarrollo	4
3.1. Paso 1: Descripción de Clases y Modelado (POO)	4
3.2. Paso 2: Implementación de <code>TreeSet</code> y Colecciones de <code>java.util</code> . . .	4
3.3. Paso 3: Ordenamiento Automático con <code>Comparable</code> e Interfaz <code>Natural</code>	5
3.4. Paso 4: Pruebas de Funcionamiento	5
3.5. Evidencias Lógicas (Algoritmos y Pseudocódigo)	6
3.5.1. Asignación de Paciente a Médico	6
3.5.2. Dar Consulta Médica	7
3.5.3. Ver Lista de Pacientes en Orden Descendente	8
3.5.4. Flujo Principal del Programa	8
4. Resultados	10
4.1. Programa Hospital	10
5. Conclusiones	24

1. Introducción

La administración de pacientes y personal médico requiere mantener organizada la información de las personas que participan en la atención hospitalaria. En este proyecto se plantea el desarrollo de un simulador de sistema hospitalario que permita registrar médicos, enfermeros y pacientes, así como establecer la relación entre ellos de acuerdo con la especialidad que requiere cada paciente.

El sistema debe considerar diferentes restricciones para representar el funcionamiento del hospital. Los médicos pueden tener hasta diez pacientes de su especialidad y atender solamente a un paciente en consulta a la vez, mientras que los enfermeros pueden tener hasta tres pacientes. Además, se requiere identificar el estado de atención de cada paciente y mostrar las listas de pacientes ordenadas alfabéticamente de forma descendente.

- **Planteamiento del problema.** Se requiere simular un sistema hospitalario que gestione el registro y la interacción entre el personal médico (médicos y enfermeros) y los pacientes. Para esta primera etapa se deben modelar cuatro clases (Sistema, Medico, Enfermero y Paciente) con los atributos y comportamientos indicados en la especificación, y aplicar las siguientes restricciones: los pacientes se asignan según la especialidad que requieren; un médico solo puede dar consulta a un paciente a la vez y tener hasta 10 pacientes de su especialidad; un enfermero puede cuidar hasta 3 pacientes a la vez de la misma especialidad; los registros se llevan en estructuras lineales; la lista de pacientes se muestra en orden alfabético descendente; y debe existir un mecanismo para saber si un paciente está en consulta o en tratamiento. La solución debe limitarse a lo visto en las cuatro prácticas del parcial.
- **Motivación.** Este problema permite aplicar, sobre un caso cercano a la realidad, los conceptos de clase, objeto, atributo, método, constructor y sobrecarga, junto con arreglos, cadenas y estructuras de control. Además, obliga a traducir reglas de negocio (límites de pacientes, coincidencia de especialidad, estado del paciente) a validaciones dentro del código, una tarea habitual en el desarrollo de software.
- **Objetivos.**
 - Desarrollar un simulador de sistema hospitalario en Java que permita registrar y administrar médicos, enfermeros y pacientes, así como asignar pacientes de acuerdo con su especialidad y controlar las acciones relacionadas con su atención.
 - Implementar las clases *Sistema*, *Medico*, *Enfermero* y *Paciente*.
 - Registrar médicos, enfermeros y pacientes mediante el sistema.
 - Asignar pacientes a médicos y enfermeros considerando su especialidad.
 - Controlar el número máximo de pacientes asignados a cada profesional.

- Permitir que un médico atienda a un solo paciente en consulta a la vez.
- Identificar si un paciente se encuentra en espera, consulta o tratamiento.

2. Marco Teórico

2.1. Entorno y lenguaje de programación

Java es un lenguaje de programación, es decir, una forma de escribir instrucciones que una computadora puede ejecutar. El programa se escribe en un archivo de texto con extensión `.java`; el compilador (`javac`) lo traduce a un formato intermedio (`.class`) y la Máquina Virtual de Java (JVM) lo ejecuta. Gracias a la JVM, el mismo programa funciona en distintos sistemas operativos.[1]

2.2. Fundamentos y sintaxis del lenguaje

La sintaxis son las reglas de escritura del lenguaje: en Java cada instrucción termina con punto y coma, los bloques se encierran entre llaves y se distingue entre mayúsculas y minúsculas. Cada dato tiene un tipo; por ejemplo, `int` guarda números enteros y `String` guarda texto. Para tomar decisiones se usan `if-else` y `switch`; para repetir instrucciones, `for` y `do-while`. La clase `Scanner` lee lo que el usuario escribe en el teclado.[1],[3]

2.3. Utilerías y clases de uso general

Un arreglo es una lista de tamaño fijo cuyos elementos son del mismo tipo; cada posición se identifica con un número que empieza en 0. Por ejemplo, un arreglo de 10 posiciones sirve para guardar hasta 10 pacientes. Un `TreeSet` es una colección que no permite elementos repetidos y ordena alfabéticamente los textos que se le agregan. Un texto es un objeto de la clase `String` y se compara con el método `equals`. [3]

2.4. Clases y objetos

Una clase es un molde que describe cómo es un tipo de elemento: qué datos tiene y qué puede hacer. Un objeto es un elemento concreto creado a partir de ese molde con la palabra `new`. Por ejemplo, la clase `Paciente` es la ficha en blanco, y cada paciente registrado es una ficha ya llena.[2]

Los atributos son los datos de un objeto (por ejemplo, el nombre de un paciente). Los métodos son las acciones que puede realizar (por ejemplo, `darConsulta`). Un objeto le pide una acción a otro escribiendo el nombre

del objeto, un punto y el método. Dentro de una clase, `this` se refiere al propio objeto.[2]

Un constructor es un método con el mismo nombre que la clase, que se ejecuta al crear un objeto y le da sus valores iniciales.[2]

La sobrecarga consiste en tener varios métodos con el mismo nombre pero con datos de entrada distintos; Java elige cuál usar según lo que recibe. En este proyecto, `asignarPaciente` existe en una versión para médicos y otra para enfermeros.[1]

3. Desarrollo

3.1. Paso 1: Descripción de Clases y Modelado (POO)

Se diseñó la arquitectura del sistema:

- **Paciente:** Almacena los datos del usuario mediante los cuales gestiona su estado. Implementando la interfaz `Comparable<Paciente>` para definir el criterio de ordenamiento natural.
- **Medico y Enfermero:** Representan al personal de salud. En esta versión, utilizan la colección `TreeSet<Paciente>` para almacenar los pacientes asignados de forma ordenada y sin duplicados.
- **Sistema:** Clase controladora encargada de coordinar los registros globales y la asignación mediante métodos sobrecargados.
- **PruebaHospital:** Contiene la interfaz de consola interactiva basada en `Scanner` y la estructura de control `switch`.

3.2. Paso 2: Implementación de `TreeSet` y Colecciones de `java.util`

En este primer enfoque se utilizó el framework de colecciones de Java para gestionar la memoria de forma dinámica:

- **Almacenamiento Dinámico:** A diferencia de los arreglos fijos, el uso de `TreeSet` permite agregar o eliminar pacientes sin necesidad de definir un tamaño estático inicial.
- **Prevención Automata de Duplicados:** `TreeSet` no permite elementos repetidos. Si se intenta insertar un paciente idéntico a uno existente, la colección lo rechaza automáticamente sin necesidad de hacer recorridos manuales con ciclos `for`.
- **Manejo de Capacidad:** Los límites máximos (10 pacientes para médicos y 3 para enfermeros) se validan directamente consultando el método `.size()` de la colección.

- **Almacenamiento Dinámico:** A diferencia de los arreglos fijos, el uso de `TreeSet` permite agregar o eliminar pacientes sin necesidad de definir un tamaño estático inicial.
- **Prevención Autómata de Duplicados:** `TreeSet` no permite elementos repetidos. Si se intenta insertar un paciente idéntico a uno existente, la colección lo rechaza automáticamente sin necesidad de hacer recorridos manuales con ciclos `for`.
- **Manejo de Capacidad:** Los límites máximos (10 pacientes para médicos y 3 para enfermeros) se validan directamente consultando el método `.size()` de la colección.

3.3. Paso 3: Ordenamiento Automático con Comparable e Interfaz Natural

En lugar de programar un algoritmo de ordenamiento manual (como Bubble Sort), el orden de los pacientes se delega totalmente a la estructura del `TreeSet`:

- **Estructura Interna:** `TreeSet` organiza sus elementos internamente mediante un **Árbol Rojo-Negro** (*Red-Black Tree*), ordenando cada elemento en el instante exacto en que se inserta con una complejidad algorítmica de $O(\log n)$.
- **Implementación de compareTo:** La clase `Paciente` define el método de la interfaz `Comparable`.
- **Despliegue de Datos:** Al invocar `verListaPacientes()`, se recorre el `TreeSet` mediante un ciclo *for-each* simple, obteniendo la lista de pacientes previamente ordenada alfabéticamente sin realizar ningún intercambio manual.

3.4. Paso 4: Pruebas de Funcionamiento

Se probó la ejecución del sistema validando las operaciones clave:

1. **Inserción y Ordenamiento:** Inserción de pacientes en orden aleatorio y verificación de que `TreeSet` los organice automáticamente al imprimirlos.
2. **Control de Duplicados:** Comprobación de que la estructura rechace elementos repetidos basándose en la respuesta devuelta por `compareTo`.
3. **Validación de Cupos:** Verificación del límite de la colección mediante `.size()` antes de cada asignación.

Diagrama de clases — Simulador de Sistema Hospitalario (Etapa 1)

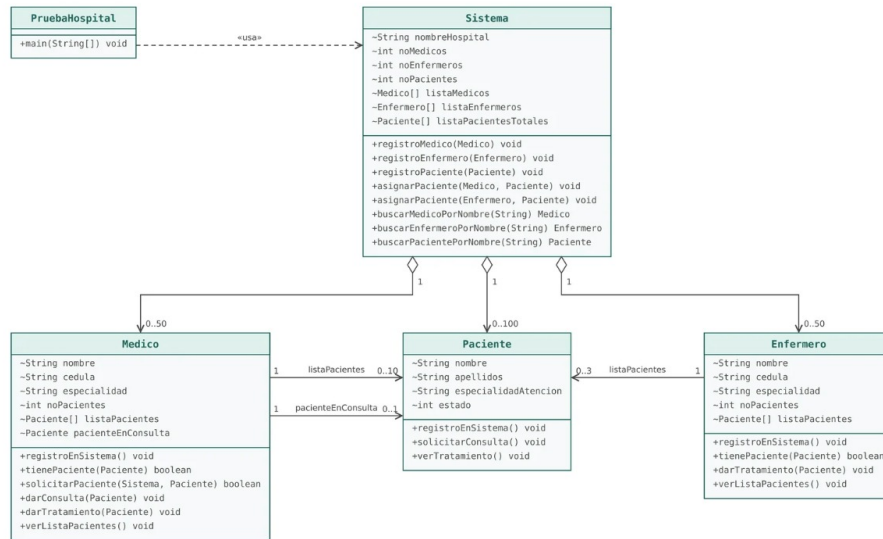


Figura 1: Diagrama de clases

3.5. Evidencias Lógicas (Algoritmos y Pseudocódigo)

3.5.1. Asignación de Paciente a Médico

Algoritmo:

1. Recibir como parámetros el objeto del médico y el objeto del paciente a asignar.
2. Validar si la especialidad del médico es diferente a la especialidad de atención que requiere el paciente. Si son diferentes, imprimir un error y terminar.
3. Verificar si el paciente ya se encuentra en la lista del médico. Si ya está, imprimir un error y terminar.
4. Comprobar si el médico ya alcanzó su límite máximo de 10 pacientes. Si es así, imprimir un error y terminar.
5. Si pasa todas las validaciones, guardar al paciente en la primera posición disponible del arreglo del médico.
6. Incrementar en uno el contador de pacientes del médico e imprimir un mensaje de éxito.

Pseudocódigo:

```

MÉTODO asignarPaciente(Medico m, Paciente p)
  SI m.especialidad NO ES IGUAL A p.especialidadAtencion ENTONCES
    IMPRIMIR "Error: La especialidad no coincide."
  
```

```

SINO SI m.tienePaciente(p) ES VERDADERO ENTONCES
    IMPRIMIR "Error: El paciente ya está asignado."
SINO SI m.noPacientes >= 10 ENTONCES
    IMPRIMIR "Error: El médico ya tiene 10 pacientes."
SINO
    m.listaPacientes[m.noPacientes] = p
    m.noPacientes = m.noPacientes + 1
    IMPRIMIR "Paciente asignado al médico"
FIN SI
FIN MÉTODO

```

3.5.2. Dar Consulta Médica

Algoritmo:

1. Recibir como parámetro al paciente que solicita la consulta.
2. Buscar al paciente en la lista asignada al médico; si no le pertenece, imprimir un error y terminar.
3. Verificar si la variable que almacena al paciente en consulta actualmente está vacía. Si no lo está, imprimir un error indicando que ya está ocupado y terminar.
4. Asignar el paciente recibido a la variable de consulta actual.
5. Cambiar el estado del paciente al valor 1 (En consulta).
6. Imprimir un mensaje indicando que la consulta ha comenzado.

Pseudocódigo:

```

MÉTODO darConsulta(Paciente p)
    SI m.tienePaciente(p) ES FALSO ENTONCES
        IMPRIMIR "Error: el paciente no está asignado."
        RETORNAR
    FIN SI
    SI pacienteEnConsulta NO ES NULO ENTONCES
        IMPRIMIR "Error: el médico ya está dando consulta a otro paciente."
        RETORNAR
    FIN SI

    pacienteEnConsulta = p
    p.estado = 1
    IMPRIMIR "El médico está dando consulta a " + p.nombre
FIN MÉTODO

```

3.5.3. Ver Lista de Pacientes en Orden Descendente

Algoritmo:

1. Inicializar una colección de tipo árbol (TreeSet) que ordenará automáticamente las cadenas alfabéticamente.
2. Iniciar un ciclo para recorrer el arreglo de pacientes desde cero hasta el total de pacientes actuales.
3. En cada iteración, extraer los apellidos, el nombre y la especialidad del paciente actual.
4. Concatenar estos datos en una sola cadena de texto y agregarla al árbol.
5. Terminado el ciclo, convertir la colección del árbol en un arreglo lineal de cadenas.
6. Iniciar un segundo ciclo que recorra este nuevo arreglo desde la última posición hasta la posición cero (recorrido inverso).
7. Imprimir cada elemento iterado para mostrar la lista en orden descendente.

Pseudocódigo:

```
MÉTODO verListaPacientes()
    CREAR TreeSet ordenadas

    PARA i DESDE 0 HASTA noPacientes - 1 CON PASO 1 HACER
        p = listaPacientes[i]
        AGREGAR (p.apellidos + " " + p.nombre + " - " + p.especialidadAtencion)
    FIN PARA

    ARREGLO arregloOrdenado = CONVERTIR ordenadas A ARREGLO

    IMPRIMIR "Lista de pacientes (Orden Descendente):"
    PARA i DESDE tamaño(arregloOrdenado) - 1 HASTA 0 CON PASO -1 HACER
        IMPRIMIR arregloOrdenado[i]
    FIN PARA
FIN MÉTODO
```

3.5.4. Flujo Principal del Programa

Algoritmo:

1. Crear una instancia del Sistema del hospital y preparar el lector de teclado (Scanner).
2. Desplegar en pantalla el menú interactivo con todas las opciones disponibles.

3. Leer el número de opción ingresado por el usuario.
4. Evaluar la opción usando una estructura de casos múltiple (switch):
 - Si es registro, pedir datos, instanciar el objeto y guardarlo en el sistema.
 - Si es atención o asignación, solicitar nombres, buscar los objetos y ejecutar el método correspondiente.
5. Repetir los pasos anteriores indefinidamente mientras la opción ingresada no sea el número cero.
6. Si la opción es cero, despedir al usuario y terminar la ejecución del programa.

Pseudocódigo:

MÉTODO main()

CREAR sistema = NUEVO Sistema("Hospital General POO")

CREAR sc = NUEVO Scanner()

HACER

IMPRIMIR menú de opciones

LEER opcion

SELECCIONAR opcion

CASO 1, 2, 3:

PEDIR datos

CREAR entidad (Medico, Enfermero o Paciente)

REGISTRAR entidad en sistema

ROMPER

CASO 4, 5:

PEDIR nombres

BUSCAR objetos en sistema

SI existen ENTONCES sistema.asignarPaciente(profesional, paciente)

ROMPER

CASO 10:

PEDIR nombres

BUSCAR medico y paciente

SI existen ENTONCES medico.darConsulta(paciente)

ROMPER

CASO 0:

IMPRIMIR "Saliendo del simulador..."

ROMPER

DEFECTO:

IMPRIMIR "Opción no válida."

FIN SELECCIONAR

```
MIENTRAS opcion SEA DISTINTO DE 0
CERRAR sc
FIN MÉTODO
```

4. Resultados

4.1. Programa Hospital

El programa es un simulador interactivo en consola que gestiona las operaciones básicas de un sistema hospitalario mediante un menú interactivo. Su objetivo principal es controlar el registro y la vinculación entre el personal médico (médicos y enfermeros) y los pacientes. El sistema valida de forma automática que las asignaciones coincidan en especialidad, respetando los límites de capacidad establecidos (10 pacientes por médico y 3 por enfermero). Además, el simulador rastrea el flujo de atención para determinar el estado de los pacientes (en espera, en consulta o en tratamiento), garantiza que los médicos atiendan a una sola persona a la vez, y genera listas de pacientes ordenadas alfabéticamente de forma descendente.

```
PS C:\Users\acer\Desktop\P00\Proyecto etapa 1 versión final> java PruebaHospital.java

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 1
Nombre del medico: Simi
Cedula: 123
Especialidad: Cardiologia
Médico Simi registrado. Especialidad: Cardiologia

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 2
Nombre del enfermero: Joy
Cedula: 456
Especialidad: Pediatria
Enfermero Joy registrado. Especialidad: Pediatria
```

Figura 2: Resultados del sistema hospitalario.

```
===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 3
Nombre del paciente: Juan
Apellidos del paciente: Zavala
Especialidad que necesita: Cardiologia
Paciente registrado: Zavala Juan

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 3
Nombre del paciente: Ana
Apellidos del paciente: Alvarez
Especialidad que necesita: Cardiologia
Paciente registrado: Alvarez Ana
```

Figura 3: Resultados del sistema hospitalario.

```
===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 3
Nombre del paciente: Luis
Apellidos del paciente: Perez
Especialidad que necesita: Pediatria
Paciente registrado: Perez Luis

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 4
Nombre del medico: Simi
Nombre del paciente a asignar: Luis
Error: La especialidad no coincide.
```

Figura 4: Resultados del sistema hospitalario.

```
===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 4
Nombre del medico: Simi
Nombre del paciente a asignar: Juan
Paciente asignado al médico Simi

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 4
Nombre del medico: Simi
Nombre del paciente a asignar: Ana
Paciente asignado al médico Simi
```

Figura 5: Resultados del sistema hospitalario.

```
===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 8
Nombre del paciente: Juan
El paciente Juan no tiene tratamiento activo.

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 10
Nombre del medico: Simi
Nombre del paciente: Juan
El médico Simi solicita al paciente Juan
El médico Simi está dando consulta a Juan
```

Figura 6: Resultados del sistema hospitalario.

```

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 10
Nombre del medico: Simi
Nombre del paciente: Ana
El médico Simi solicita al paciente Ana
Error: el médico Simi ya está dando consulta a Juan. Solo puede atender a un paciente a la vez.

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 11
¿Quién dará el tratamiento? (1=Medico, 2=Enfermero): 1
Nombre del profesional: Simi
Nombre del paciente: Juan
El médico receta tratamiento a Juan

```

Figura 7: Resultados del sistema hospitalario.

```
===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 8
Nombre del paciente: Juan
El paciente Juan está recibiendo tratamiento.

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 6
Nombre del medico: Simi

Lista de pacientes de Simi (Orden Descendente):
Zavala Juan - Cardiologia
Alvarez Ana - Cardiologia
```

Figura 8: Resultados del sistema hospitalario.

```

PS C:\Users\acer\Desktop\P00\Proyecto etapa 1 versión final> java PruebaHospital

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 2
Nombre del enfermero: Joy
Cedula: 456
Especialidad: Pediatria
Enfermero Joy registrado. Especialidad: Pediatria

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 3
Nombre del paciente: Luis
Apellidos del paciente: Zavala
Especialidad que necesita: Pediatria
Paciente registrado: Zavala Luis

```

Figura 9: Resultados del sistema hospitalario.

```
===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 3
Nombre del paciente: Ana
Apellidos del paciente: Alvarez
Especialidad que necesita: Pediatria
Paciente registrado: Alvarez Ana

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 3
Nombre del paciente: Barry
Apellidos del paciente: Gomez
Especialidad que necesita: Pediatria
Paciente registrado: Gomez Barry
```

Figura 10: Resultados del sistema hospitalario.

```
===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 3
Nombre del paciente: Carlos
Apellidos del paciente: Ruiz
Especialidad que necesita: Pediatria
Paciente registrado: Ruiz Carlos

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 3
Nombre del paciente: Juan
Apellidos del paciente: Perez
Especialidad que necesita: Cardiologia
Paciente registrado: Perez Juan
```

Figura 11: Resultados del sistema hospitalario.

```
===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 5
Nombre del enfermero: Joy
Nombre del paciente a asignar: Juan
Error: La especialidad no coincide.

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 5
Nombre del enfermero: Joy
Nombre del paciente a asignar: Luis
Paciente asignado al enfermero Joy
```

Figura 12: Resultados del sistema hospitalario.

```
===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 5
Nombre del enfermero: Joy
Nombre del paciente a asignar: Ana
Paciente asignado al enfermero Joy

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 5
Nombre del enfermero: Joy
Nombre del paciente a asignar: Barry
Paciente asignado al enfermero Joy
```

Figura 13: Resultados del sistema hospitalario.

```
===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 5
Nombre del enfermero: Joy
Nombre del paciente a asignar: Carlos
Error: El enfermero ya tiene 3 pacientes.

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 8
Nombre del paciente: Ana
El paciente Ana no tiene tratamiento activo.
```

Figura 14: Resultados del sistema hospitalario.

```

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 11
¿Quien dara el tratamiento? (1=Medico, 2=Enfermero): 2
Nombre del profesional: Joy
Nombre del paciente: Ana
El enfermero Joy aplica el tratamiento a Ana

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 8
Nombre del paciente: Ana
El paciente Ana está recibiendo tratamiento.

```

Figura 15: Resultados del sistema hospitalario.

```

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 7
Nombre del enfermero: Joy

Lista de pacientes de Joy (Orden Descendente):
Zavala Luis - Pediatria
Gomez Barry - Pediatria
Alvarez Ana - Pediatria

===== MENU HOSPITAL =====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
0. Salir
Elige una opcion: 0
Saliendo del simulador...
PS C:\Users\acer\Desktop\P00\Proyecto etapa 1 versión final> |

```

Figura 16: Resultados del sistema hospitalario.

5. Conclusiones

El desarrollo del simulador de sistema hospitalario permitió aplicar los principales conceptos de Programación Orientada a Objetos en Java. Mediante las clases *Sistema*, *Medico*, *Enfermero* y *Paciente* se representaron de forma organizada los elementos principales del sistema.

Durante la implementación se utilizaron arreglos como estructuras lineales, ciclos para búsquedas y recorridos, sobrecarga de métodos y estructuras de ordenamiento. También se implementó un mecanismo para identificar si un paciente se encuentra en espera, consulta o tratamiento.

Finalmente, el proyecto permitió integrar los conocimientos adquiridos durante el parcial y comprender cómo las clases, objetos y estructuras de datos pueden utilizarse para resolver un problema de forma organizada.

Referencias

- [1] M. A. Nakayama C. y J. A. Solano Gálvez, *Manual de prácticas del Laboratorio de Modelos de Programación Orientada a Objetos (MADO-21)*, versión 02. Ciudad de México: Facultad de Ingeniería, UNAM, 2018.
- [2] L. Joyanes Aguilar, *Fundamentos de programación. Algoritmos, estructuras de datos y objetos*, 4.^a ed. México: McGraw-Hill, 2008.
- [3] J. Dean y R. Dean, *Introducción a la programación con Java*, 1.^a ed. México: McGraw-Hill, 2009.